Lecture 03 NOTES – Process Creation, Termination & Threads Introduction

What is POSIX?

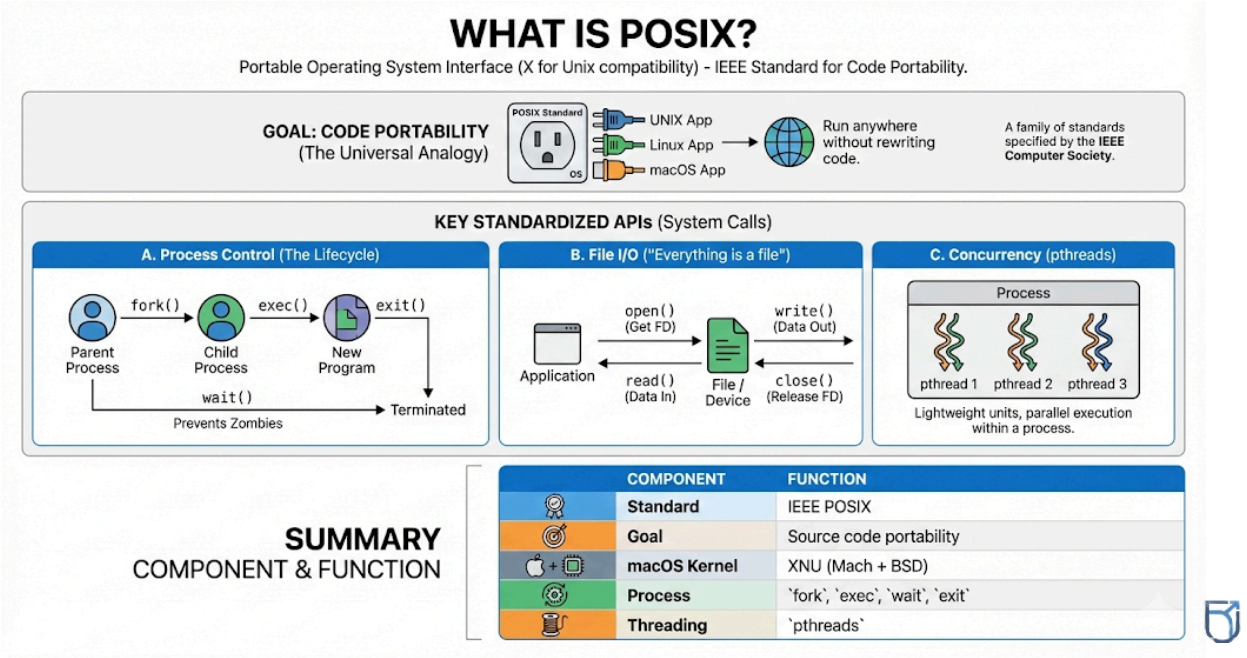
- **Full Form: P**ortable **O**perating **S**ystem **I**nterface (the **X** stands for API compatibility with Unix).
- **Definition:** A family of standards specified by the IEEE Computer Society.
- **Primary Goal: Code Portability.** It ensures that software written for one UNIX-like OS can run on another without rewriting the code.
    - *Analogy:* Like a universal power outlet; if the OS fits the standard, the "app plug" will work.

Key Standardized APIs (System Calls)

POSIX standardizes how applications talk to the Kernel.

A. Process Control (The Lifecycle)

- fork(): Creates a new process by duplicating the calling process (Parent to Child).
- exec(): Replaces the current process image with a new program (runs new code).
- wait(): Makes the parent process pause until the child process finishes (prevents "zombie" processes).
- exit(): Terminates the process and returns a status code to the OS.



B. File I/O (Input/Output)

- *Philosophy:* "Everything is a file."
- open(): Opens a file and returns a **File Descriptor** (integer ID).
- read() / write(): core data transfer functions.
- close(): Releases the file descriptor.

C. Concurrency

- **pthreads** (POSIX Threads): The standard API for creating and managing threads (lightweight processes) in C/C++.

Component	**Function**
Standard	IEEE POSIX
Goal	Source code portability
macOS Kernel	XNU (Mach + BSD)
Process	fork, exec, wait, exit
Threading	pthreads

Process Creation — `fork()`

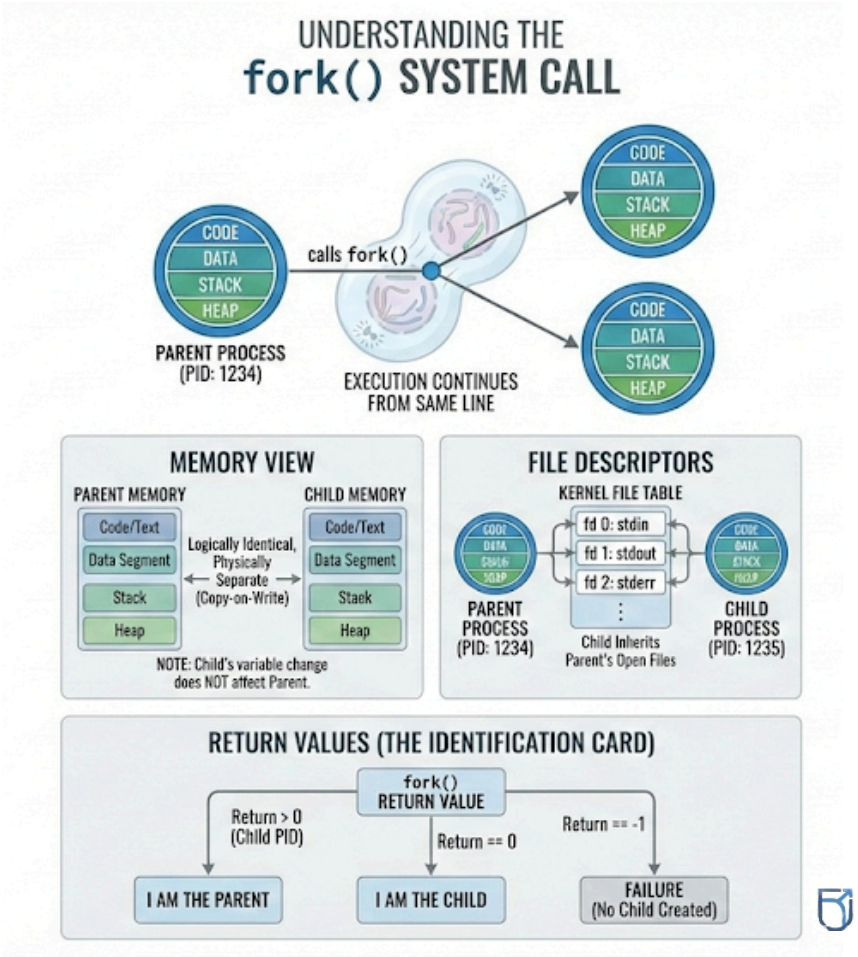
Definition

- **What is it?** A primary POSIX system call (unistd.h).
- **Action:** It creates a **new process** (Child) by duplicating the **calling process** (Parent).
- **Analogy:** Biological cell division (Mitosis) — one cell divides into two identical copies.

Behavior: "The Clone"

When fork() is called, the OS creates an exact copy of the parent.

- **Code & Data:** Child gets a copy of the Parent's code, data segment, stack, and heap.
    - *Note:* They are **logically identical** but **physically separate**. If the Child changes a variable, the Parent's variable *does not* change.
- **File Descriptors:** Child inherits the Parent's open files (points to the same file table entries).



- **Execution:**
    - Execution continues from the **exact same line** (the instruction immediately after fork()) in *both* processes.
    - They run **concurrently** (OS scheduler decides who runs first).

Return Values (The Identification Card)

Condition	Return Value	Meaning
Child Process	0	"I am the new child."

Parent Process	> 0 (Child's PID)	"I am the parent; here is my child's ID."
Failure	-1	Creation failed (no child created).

Memory Model After fork()

When fork() creates a child process, it creates an **illusion** of a separate memory space, but the reality is optimized by the hardware.

- **Logical View (The Programmer's Perspective):**
  - The Child appears to have an exact, independent copy of the Parent's memory.
  - It contains identical **Text** (code), **Data** (global vars), **Heap** (dynamic alloc), and **Stack** (local vars).
- **Physical View (The OS Perspective):**
  - The OS does **not** blindly copy every byte of RAM immediately. That would be too slow. Instead, it uses **COW**.

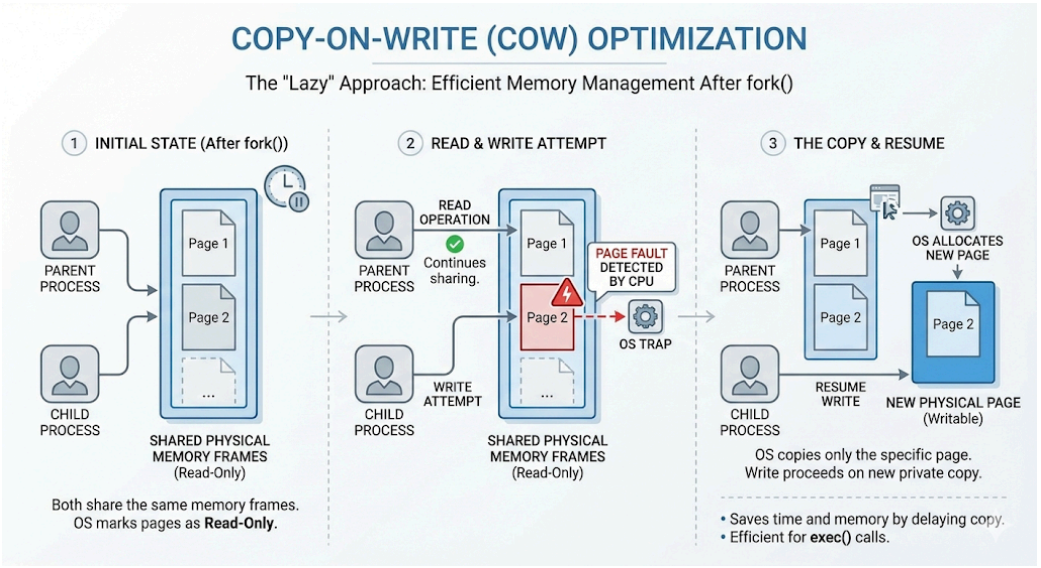
```c
#include <stdio.h>    // Required for printf
#include <unistd.h>   // Required for fork() and sleep()
#include <sys/types.h> // Required for pid_t
#include <stdlib.h> // Required for pid_t exit()
int main() {
    int x = 10;
    // fork() returns pid_t (a type of integer)
    pid_t pid = fork();
    if (pid < 0) {
    // perror automatically appends the specific reason for failure
    perror("fork failed");
    exit(1); }
    else if (pid == 0) {  // Child process
        x = 20;
        printf("Child x = %d\n", x);
    }
    else {               // Parent process
        // Parent sleeps to allow child to potentially execute first
```

```
        sleep(1);
        printf("Parent x = %d\n", x);
    }
    return 0;
}
```

Copy-On-Write (COW) Optimization

Why do we need it?

- **Expense:** Copying the entire address space (e.g., a 4GB browser process) takes time and doubles memory usage instantly.
- **Waste:** In 99% of cases, the Child calls exec() immediately after fork(). exec() wipes the memory to load a new program, meaning the previous copy would have been completely wasted effort.



Code:

```
#include <unistd.h>
#include<stdio.h>
int main(){
    int x = 100;
    pid_t pid = vfork();
    if (pid == 0) {
        x = 200;        // overwrites parent's variable!
        _exit(0);
```

```
    }
    printf("parent x = %d\n", x); // Output: 200 (corrupted)
    return 0;
}
```

Output

```
parent x = 100
```

How COW Works (The "Lazy" Approach):

1. **Initial State:** After fork(), Parent and Child share the **same physical memory frames**. The OS marks these pages as **Read-Only**.
2. **Read Operations:** If both just *read* data, they continue sharing the same physical RAM. No copying happens.
3. **Write Attempt:** If either process tries to *modify* (write) data (e.g., changing a variable):
    - The CPU detects a write to a "Read-Only" page and triggers a **page fault**.
    - The OS catches this fault.
4. **The Copy:** The OS allocates a **new physical page**, copies *only* that specific page, and maps it to the writing process.
5. **Resume:** The write operation proceeds on the new private copy.

Segment-Wise Behavior:

Segment	**Function**	**Behavior after fork()**
Text	Code Instructions	**Shared Always** (Since code typically doesn't change, it remains read-only and shared).

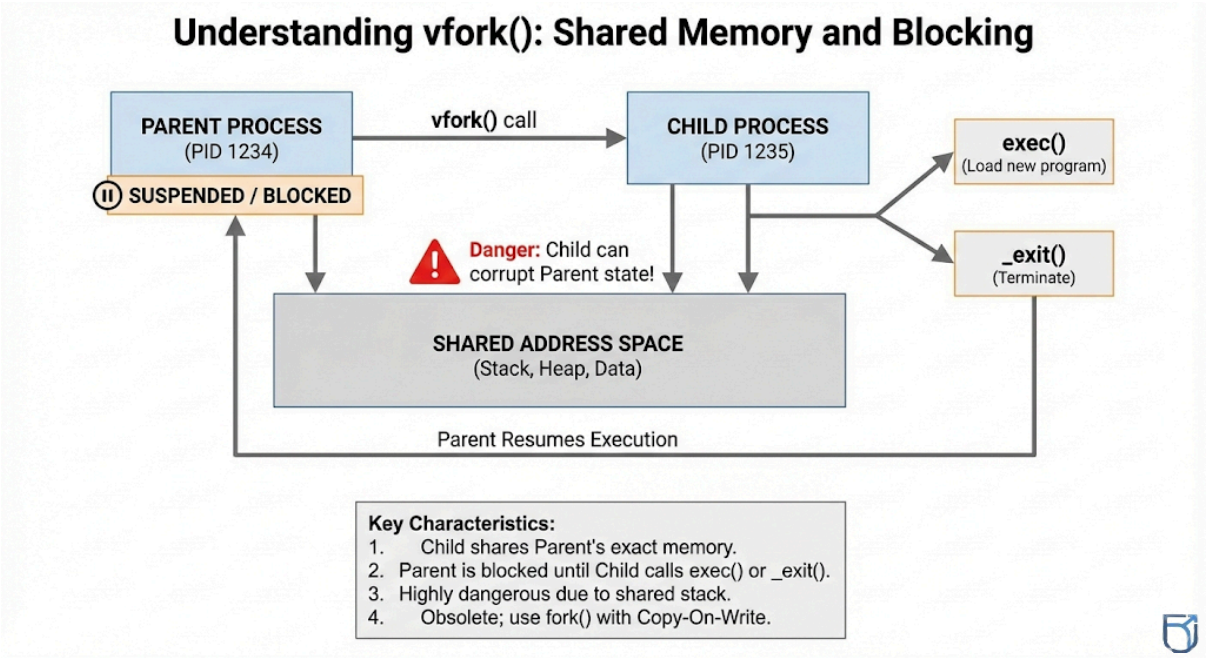
Data	Global/Static Vars	**COW** (Shared until one process modifies a variable).
Heap	Dynamic Memory	**COW** (Shared until malloc/free or data change occurs).
Stack	Local Variables	**COW** (Shared until a variable changes or a function is pushed/popped).

Advantages of COW:

- **Speed:** Process creation is almost instantaneous because no data is actually copied at start.
- **Memory Efficiency:** Reduces RAM usage significantly, especially for processes that fork often (like shell or web servers).

vfork() (Obsolete & Dangerous)

- **Status: Deprecated/Obsolete** (marked as such in macOS and modern Linux standards).
- **Concept:** Originally designed as a "faster fork" before Copy-On-Write (COW) was invented.



- **Behavior:**
  - **No Copying:** It does *not* copy the address space.
  - **Shared Memory:** The Child runs in the **exact same memory space** as the Parent (sharing Stack, Heap, Data).
  - **Parent Suspended:** The Parent process is paused (blocked) until the Child calls exec() or _exit().
- **The Danger Zone:**
  - Since they share the stack, if the Child changes a variable or pops a stack frame, **the Parent sees that change** when it wakes up.
  - **High Risk:** It is extremely easy for the Child to corrupt the Parent's memory state.
- **Verdict: Do not use.** Modern COW makes standard fork() fast enough, rendering vfork() unnecessary and risky.

1. What is `exec()`?

The `exec()` system call is used to **run a new program**. It replaces the currently running program with a completely new one.

- **Core Concept:** It is a "Brain Transplant" for a process.
- **No New Process:** It does **not** create a new process ID (PID). The PID remains the same, but the code, data, and stack are replaced.
- **One-Way Ticket:** Once `exec()` is successful, the old code is gone forever. It cannot return to the old program

Purpose:

To replace the current process's memory image (code, data, stack) with a new program.

Key Characteristics:

- **Transformation:** It does *not* create a new process. It transforms the existing one.
- **Identity:** The **PID remains the same**. The OS just swaps the "brain" (code) of the process.
- **Return Value:** exec() functions **never return** to the caller if they are successful (because the code that called them is gone!). They only return -1 if they fail.

Difference: fork() vs exec()

Feature	**fork()**	**exec()**

Action	Creates a **New** Process (Clone).	Replaces the **Current** Process (Transform).
PID	Generates a new PID for the child.	Retains the existing PID.
Memory	Copies parent's memory (COW).	Overwrites memory with new program.
Analogy	Cell Division (Mitosis).	Brain Transplant (New personality, same body).

Why is `exec()` Needed? (The Shell Workflow)

When you type a command like `./a.out` or `ls` in your terminal, the OS follows a specific flow:

1. Fork: The Shell calls `fork()` to create a child process.
2. Exec: The Child calls `exec("./a.out")`. This replaces the Shell's code in the child with the code of `a.out`.
3. Wait: The original Shell (Parent) stays alive and waits for the child to finish.

Impact on Process Memory

When `exec()` runs, some things are wiped out, and others are kept.

The `exec()` Family of Functions

There is no single function called just `exec`. There is a family of functions acting as wrappers. `execve()` is the actual System Call; the rest are library functions.

Naming Convention (The Secret Code):

- **l (list):** Arguments are passed one by one (`arg0`, `arg1`, `...`, `NULL`).
- **v (vector):** Arguments are passed as an array (vector).
- **p (path):** The OS searches the system `PATH` variable (you can type `"ls"` instead of `"/bin/ls"`).

- **e (env):** You provide a custom environment variable array.

Types of exec() family functions :

1. execl() (List, Manual Path)

- **How it works:** You list the arguments one by one (comma-separated). You must provide the **full file path** (e.g., `/bin/ls`).
- **Use when:** You know the exact location of the file and the exact arguments at compile time.
- **Syntax:** `execl("/bin/ls", "ls", "-l", NULL);`

2. execlp() (List, Use PATH)

- **How it works:** Same as `execl` (arguments listed one by one), but you only give the **filename** (e.g., `ls`). The OS searches the system `PATH` variable to find it.
- **Use when:** You want to run standard commands like `ls`, `grep`, or `python` without knowing exactly where they are installed.
- **Syntax:** `execlp("ls", "ls", "-l", NULL);`

3. execle() (List, Custom Environment)

- **How it works:** Like `execl`, but allows you to pass a **custom environment** array (variables like `HOME`, `PATH`) instead of inheriting the current one.
- **Use when:** You want to run a program in a secure or modified environment (sandbox).
- **Syntax:** `execle("/bin/ls", "ls", "-l", NULL, my_env_array);`

---

4. execv() (Vector, Manual Path)

- **How it works:** You pass arguments as an **array (vector)** of strings. You must provide the **full file path**.
- **Use when:** You are building a shell or a program where the number of arguments is dynamic (not known until runtime).
- **Syntax:** `execv("/bin/ls", my_args_array);`

5. execvp() (Vector, Use PATH)

- **How it works:** Same as `execv` (arguments in an array), but the OS searches the system `PATH` to find the program.
- **Use when:** You want to run standard commands with a dynamic list of arguments. This is the **most common** choice for writing shell programs.
- **Syntax:** `execvp("ls", my_args_array);`

6. execve() (Vector, Custom Environment)

- **How it works:** This is the **actual system call**. All others are just wrappers around this. It takes arguments in an array AND a custom environment array.
- **Use when:** You need full, low-level control over everything (arguments and environment).
- **Syntax:** `execve("/bin/ls", my_args_array, my_env_array);`

The Comparison Table:

Function	Arguments	Uses PATH?	Environment	Notes
execl()	List	❌ No	Inherited	Manual argument list.
execlp()	List	✅ Yes	Inherited	Most common "List" version.
execle()	List	❌ No	**Custom**	Allows custom environment.
execv()	Vector	❌ No	Inherited	Uses an array of strings.
execvp()	Vector	✅ Yes	Inherited	Most common "Vector" version.
execve()	Vector	❌ No	**Custom**	**The Real System Call.**

Implementation Examples :-

1-execl() System Call :

```c
#include <unistd.h>
#include <stdio.h>
int main() {
    printf("Before exec()\n");
    execl("/bin/ls", "ls", NULL);
    printf("This will NOT print if exec is successful\n");
}
```

NOTE: The second printf never runs — because exec() replaced the program.

Output

```
Before exec()
b.out           fork1           sec.c
e2               fork1.c         tempCodeRunnerFile
e2.c            forkcomplete    tempCodeRunnerFile.c
exee.out        forkcomplete.c  vfork
exee1           forkcow         vfork.c
exee1.c         forkcow.c       ww
first           hello.txt       ww.c
first.c         sec
```

2-execl() System Call :

//child.c

```c
#include <stdio.h>
int main(){
    printf("Child executed successfully!\n");
    return 0;
}
```

//e3.c

```c
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {                          // Child process
        execl("./child", "child", NULL);     // Executes the child
program
        printf("after execl()......");       // Will run ONLY if execl
fails
        perror("exec failed");               // Only prints if execl
fails
        _exit(1);
    } else {                                 // Parent process
        wait(NULL);                          // Wait for child to
finish
        printf("Parent process finished.\n");
    }
}
```

Output

```
Child executed successfully!
Parent process finished.
```

The "Fork-Exec" Model (Shell Behavior)

Why do we need both? This is how your Terminal (Shell) runs commands like ls or python.

1. **The Fork:** The Shell calls fork() to create a Child process.
    - *Now we have two Shells running.*

2. **The Decision:**
    - **Parent (Shell):** Calls wait() to pause and let the command run.
    - **Child (Shell copy):** Immediately calls exec("ls").
3. **The Transformation:**
    - The Child acts as a wrapper. The exec() call replaces the "Child Shell" code with the "ls" binary code.
    - The PID stays the same, but now it is running ls instead of bash/zsh.
4. **Completion:**
    - ls finishes and exits.
    - Parent (Shell) wakes up from wait() and gives you the prompt back.

execve

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
   pid_t pid = fork();
   if (pid == 0) {
       // Child process - replace with ls command
       char *args[] = {"/bin/ls", "-l", "/tmp", NULL};
       char *env[] = {NULL};

       printf("Child about to exec...\n");
       execve("/bin/ls", args, env);

       // If we reach here, exec failed
       perror("execve failed");
       return 1;
   } else {
       // Parent waits for child
       wait(NULL);
       printf("Parent waiting for child %d\n", pid);
```

```
        printf("Parent: child finished\n");
    }
    return 0;
}
```

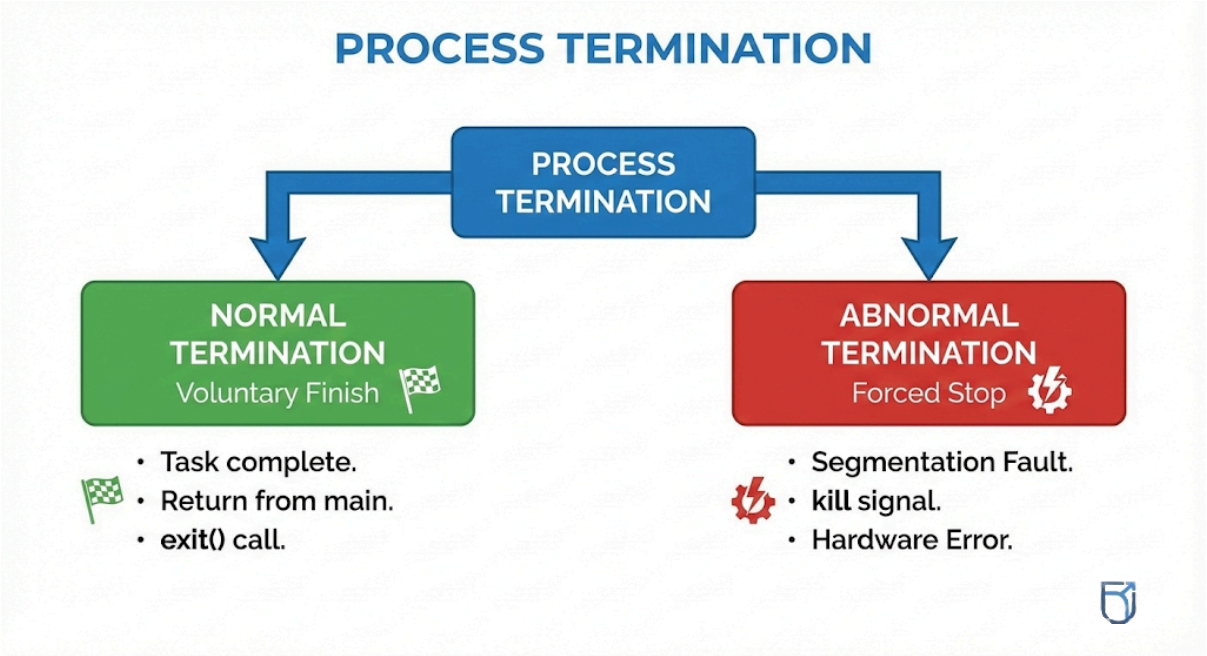
Output

```
Child about to exec ...
lrwxr-xr-x@ 1 root  wheel  11 Aug 17 00:14 /tmp → private/tmp
Parent waiting for child 2657
Parent: child finished
```

Process Termination

A process can end in two ways:

- **Normal Termination:** The process finishes its task voluntarily.
- **Abnormal Termination:** The process is forced to stop (e.g., Segmentation Fault, kill signal, or external hardware error).



Normal Termination (exit)

How it happens:

1. return n from main().
2. Explicitly calling exit(status).

What exit() Does (The "Polite" Exit):

exit() is a high-level C Library function (stdlib.h). It performs housekeeping before handing control to the OS.

1. **Callbacks:** Executes functions registered with atexit().
2. **I/O Cleanup: Flushes** standard I/O buffers (writes pending data to files).
3. **Resources:** Closes all open streams (files).
4. **Final Step:** Calls the kernel system call _exit().

_exit() System Call

Definition:

_exit() is the low-level POSIX System Call (unistd.h) that actually triggers the kernel to destroy the process.

What it Does (The "Immediate" Exit):

It terminates the process immediately.

What it Does NOT Do:

- Does **NOT** flush stdio buffers.
- Does **NOT** call atexit() handlers.
- Does **NOT** perform user-space cleanup.

Critical: Why use _exit() in a Child Process?

When you fork(), the Child inherits the Parent's buffered data.

- **The Scenario:**
    1. Parent calls printf("Hello"). The data goes into a buffer (not printed to screen yet).
    2. fork() happens. Child inherits the buffer containing "Hello".
    3. If Child calls exit() (the normal one): It flushes the buffer and prints "Hello".
    4. Later, Parent calls exit(): It *also* flushes its buffer and prints "Hello".
- **The Result:** "Hello" appears on the screen **twice**.

The Solution:

- **Child must use _exit():** This drops the process immediately without touching the buffers.
- **Parent uses exit():** This performs the cleanup and flushing normally.

Comparison Table

Feature	exit()	_exit()
Type	C Library Function	POSIX System Call
Cleanup	Flushes buffers, runs handlers	No cleanup, immediate death
Use Case	Normal program end	Inside fork() child code
Header	<stdlib.h>	<unistd.h>

_exit() : Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void cleanup1() {
    printf("Cleanup handler 1 executed\n");
}

void cleanup2() {
    printf("Cleanup handler 2 executed\n");
}

void cleanup3() {
    printf("Cleanup handler 3 executed\n");
}
```

```c
int main() {
    printf("Program started (PID: %d)\n", getpid());

    // Register cleanup handlers
    atexit(cleanup1);  // Registered first
    atexit(cleanup2);  // Registered second
    atexit(cleanup3);  // Registered third

    printf("Main program logic executing\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child process (PID: %d)\n", getpid());
        sleep(1);

        // IMPORTANT: child exits using _exit()
        _exit(0);
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: Child completed\n");
        printf("Parent calling exit()\n");

        // Parent terminates normally
        exit(0);
    }

    // Never reached
```

```
    printf("This line never executes\n");
    return 0;
}
```

Abnormal Termination

Sometimes processes die unexpectedly. This is usually caused by **Signals** sent by the OS or other users.

- **Default Action:** Most signals cause the process to dump core (save memory state to disk for debugging) and terminate immediately.
- **Common Signals Table:**

Abnormal Termination Example

Signal	Description	Can be Caught?
SIGINT	Triggered by Ctrl+C. Interrupts the process.	✅ Yes
SIGTERM	"Please stop." A polite request to terminate. allows cleanup.	✅ Yes
SIGKILL	"Die immediately." Forced murder. No cleanup allowed.	**NO**
SIGSEGV	Segmentation Fault. Triggered by illegal memory access.	Yes (but risky)

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child - infinite loop
        printf("Child running (PID: %d)\n", getpid());
        while(1) {
            sleep(1);
        }
    }
    else {
        // Parent
        sleep(2);
        printf("Parent sending SIGKILL to child\n");
        kill(pid, SIGKILL);  // Abnormal termination
        wait(NULL);
        printf("Child terminated abnormally\n");
    }
    return 0;
}
```

Output:

```
Child running (PID: 100878)
Parent sending SIGKILL to child
Child terminated abnormally
```

wait() System Call

Purpose:

The "reaper" function. The Parent process must acknowledge the death of its Child to release the Child's resources from the system table.

Behavior:

1. **Blocking:** When the Parent calls wait(), it pauses execution (sleeps).
2. **Waiting:** It remains blocked until **any one** of its children terminates.
3. **Collection:** Once a child dies, the OS wakes the Parent. wait() returns the deceased Child's PID and stores its exit status (return code) in an integer variable.
4. **Zombie Prevention:** Calling wait() prevents the child from becoming a **Zombie Process** (a dead process taking up space in the process table).

Syntax:

pid_t child_pid = wait(&status);

- **Returns:** PID of the finished child (Success) or -1 (Error/No children left).

waitpid() — The Upgrade

wait() is simple, but it has flaws: it blocks *everything* and waits for *anyone*. waitpid() offers precision control.

Advantages over wait():

1. **Specific Targeting:** You can wait for a **specific child** process (by PID) instead of just "the first one to die."
2. **Non-Blocking Option:** By using the WNOHANG flag, the Parent can check if a child is dead. If the child is still running, waitpid() returns 0 immediately instead of freezing the Parent.

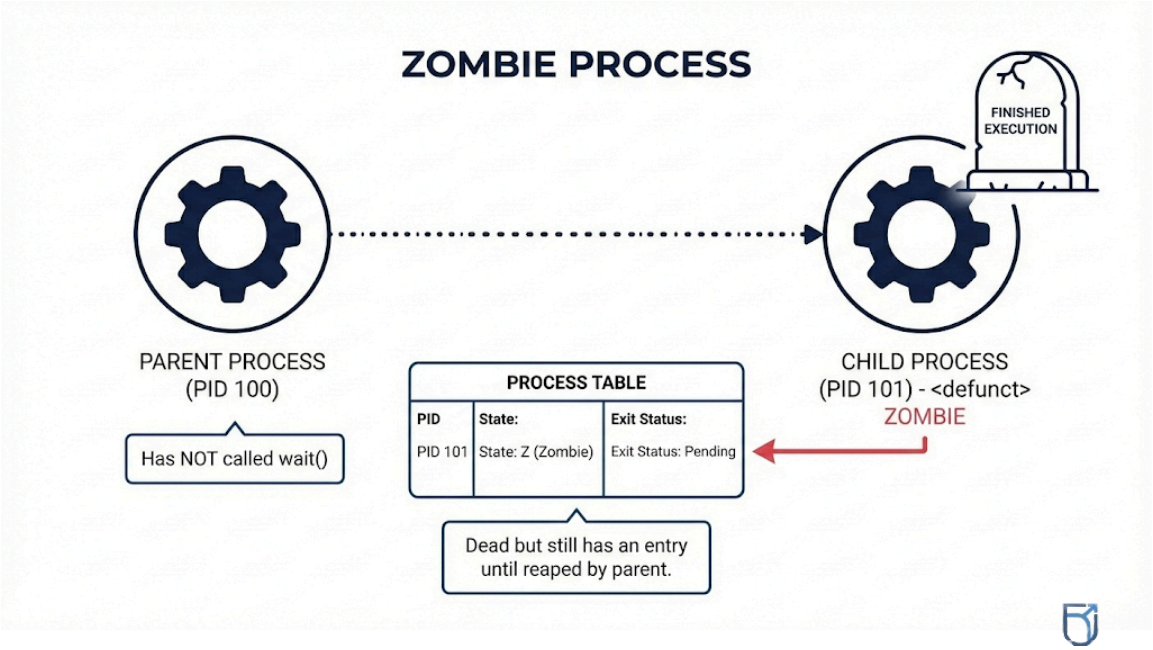
Comparison Table:

Feature	**wait()**	**waitpid()**
Target	Waits for **any** child.	Can wait for a **specific** PID.
Blocking	Always blocks until a child exits.	Can be **Non-Blocking** (WNOHANG).

Control	Basic.	Advanced (supports job control).

Zombie Process (The "Undead")

- **Definition:** A child process that has finished execution (exit()), but its parent has **not yet** called wait() to read its exit status.
- **State:** It is dead code-wise but "alive" in the system table.



- **Characteristics:**
  - **Resources:** Releases all memory and file descriptors (0 resources used).
  - **Remains:** Only a tiny entry in the **Process Control Block (PCB)** remains (PID + Exit Status).
  - **Identification:** Shows as Z or <defunct> in ps or top commands.
- **The Problem:** If too many zombies accumulate, the system runs out of PIDs (Process IDs), preventing new processes from starting.
- **Solution:** The Parent **must** call wait() or waitpid() to "reap" the zombie and remove the entry.

Zombie Processes : Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```c
int main() {
    printf("Parent PID: %d\n", getpid());
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {  // Child process
        printf("Child PID: %d\n", getpid());
        printf("Child: I'm done, exiting...\n");
        exit(0);  // Child exits immediately
    }
    // Parent process
    printf("Parent: NOT calling wait()\n");
    printf("Parent: Child %d is now a ZOMBIE!\n", pid);
    printf("Parent: Run 'ps aux | grep Z' to see it\n");
    sleep(30); // Parent sleeps so zombie stays visible
    printf("Parent: Exiting (zombie will be cleaned by init)\n");
    return 0;
}
```

Output

```
Parent PID: 63704
Parent: NOT calling wait()
Child PID: 63705
Parent: Child 63705 is now a ZOMBIE!
Parent: Run 'ps aux | grep Z' to see it
Child: I'm done, exiting...
Parent: Exiting (zombie will be cleaned by init)
```

Orphan Process (The "Abandoned")

- **Definition:** An orphan process is a running child process whose **parent has finished or terminated** before the child could finish.
- **The Scenario:**
    - A Parent creates a Child (`fork()`).
    - The Parent finishes its task and exits.

- The Child is still running (e.g., doing a long calculation or waiting for I/O).
- The Child is now an "Orphan" because it has no parent to report to.

2. How the OS Handles Orphans

The Operating System does not leave orphans floating around. It has a specific protocol:

- **Detection:** The OS notices a process has lost its parent.
- **Adoption:** The orphan is immediately adopted by the "Root" process of the system.
  - **Linux:** Adopted by `init` or `systemd` (PID 1).
  - **macOS:** Adopted by `launchd` (PID 1).
- **The Result:** The orphan now has a new parent (PID 1).

3. Is it Harmful? (The Verdict)

- **No, Orphans are safe.**
- **Why?** The new parent (`init`/`systemd`) is programmed to be a "responsible parent." It automatically calls `wait()` whenever one of its adopted children finishes.
- **Cleanup:** Because PID 1 waits for them, the orphan is properly cleaned up (reaped) when it finishes. It does **not** become a permanent zombie or cause memory leaks.

Orphan Process Example:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h> // Required for wait()
int main() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {  // Child process
        printf("Child: My parent PID: %d\n", getppid());
        printf("Child: Working for 5 seconds...\n");
```

```
sleep(5);
        printf("Child: Finished work. Exiting...\n");
exit(0);
    }
    else {  // Parent process
        printf("Parent: Waiting for my child (PID %d) to
finish...\n", pid);
        // SOLUTION: The parent blocks here and waits for the child
        int status;
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child finished normally with exit code
%d.\n", WEXITSTATUS(status));
        }
        printf("Parent: Child is reaped. Parent is now exiting
safely.\n");
        exit(0);
    }
    return 0;
}
```

Output

```
Parent: Waiting for my child (PID 62840) to finish...
Child: My parent PID: 62839
Child: Working for 5 seconds...
Child: Finished work. Exiting...
Parent: Child finished normally with exit code 0.
Parent: Child is reaped. Parent is now exiting safely.
```